CS336 Lecture 2 学习讲义

> 副标题：从张量、内存、计算量，到模型、优化器与训练循环
>
> 适用对象：正在上 CS336 的课程同学
>
> 讲义依据：lecture/lec2.py

目录

1. 这节课到底在解决什么问题
2. 资源视角：训练模型时我们究竟在消耗什么
3. 张量基础：训练系统的一切都从 tensor 开始
4. 内存核算：dtype、参数、梯度、激活值到底占多少
5. 张量存储与视图：stride、view、transpose、contiguous
6. 计算核算：为什么矩阵乘法主导训练成本
7. `einops` 与 `jaxtyping`：让高维张量代码更可读
8. 自动求导与反向传播：`loss.backward()` 到底做了什么
9. 模型与参数：`nn.Parameter`、初始化、自定义 `nn.Module`
10. 训练工程基础：随机性、数据加载、优化器、训练循环
11. checkpoint 与混合精度训练
12. 本讲总结与复习清单

1. 这节课到底在解决什么问题

这节课的核心不是 Transformer 结构细节，而是更底层的训练原语：

- 如何表示数据和参数
- 如何估算内存消耗
- 如何估算计算量
- 如何组织前向、反向、参数更新
- 如何把这些组件串成一个最小可训练系统

换句话说，这节课在回答一个很工程的问题：

> 如果你现在要训练一个模型，你必须会哪些最基本的东西？

原讲义强调了两个贯穿始终的视角：

- `Memory`：你的参数、梯度、优化器状态、激活值一共占多少显存
- `Compute`：你每一步训练需要做多少 FLOPs，硬件每秒能做多少 FLOP/s

这两个问题几乎决定了训练是否可行。

2. 资源视角：训练模型时我们究竟在消耗什么

2.1 两个最重要的资源

训练大模型时，最关键的不是“代码能不能跑”，而是：

- 放不放得下
- 跑不跑得动

更具体地说：

资源	典型单位	决定因素
内存	Bytes / MB / GB	参数量、激活值、梯度、优化器状态、dtype
计算	FLOPs	模型大小、token 数、矩阵乘法规模

2.2 讲义开头的两个估算题

原讲义一上来就做了两个 back-of-the-envelope calculation，这非常重要，因为训练系统设计常常先靠估算再靠实验。

问题 1：训练一个 70B 参数模型，吃 15T token，用 1024 张 H100，要多久？

讲义里用了这个近似公式：

$$\text{total FLOPs} \approx 6 \times \text{参数量} \times \text{token 数}$$

于是：

$$6 \times 70 \times 10^9 \times 15 \times 10^{12} = 6.3 \times 10^{24} \text{ FLOPs}$$

如果一张 H100 的有效吞吐按讲义中的近似取值：

- 理论峰值： `1979e12 / 2` FLOP/s
- 再乘一个经验上的 `MFU = 0.5`

那么 1024 张卡每天能提供的总 FLOPs 约为：

$$\text{H100 FLOP/s} \times 0.5 \times 1024 \times 86400$$

最后得到训练时间大约是数十天量级。

这类估算的价值在于：

- 帮你判断一个项目是“几天实验”还是“几周甚至几个月工程”
- 帮你在模型规模、数据规模、硬件规模之间做 trade-off

问题 2：8 张 H100，朴素使用 AdamW，最多能训练多大的模型？

讲义里的简化核算思路是：

- 参数本身： `4 bytes`
- 梯度： `4 bytes`
- 优化器状态： `4 + 4 bytes`

所以每个参数大约占：

$$4 + 4 + (4 + 4) = 16 \text{ bytes}$$

8 张 H100，每张 80 GB，则总显存约：

$$8 \times 80 \text{ GB} \,/\, 16 \text{ bytes per parameter}$$

得到的参数量大约是数百亿级别。

这只是非常粗糙的上界，因为还没有算：

- 激活值
- 临时 buffer
- 通信开销
- framework overhead

但这已经足够让你形成一个工程直觉：

> 显存限制往往不是“模型参数”单独决定的，而是“参数 + 梯度 + 优化器状态 + 激活值”一起决定的。

3. 张量基础：训练系统的一切都从 tensor 开始

在 PyTorch 里，几乎所有核心对象最终都是 tensor：

- 输入数据
- 模型参数
- 梯度
- 优化器状态
- 中间激活值

3.1 创建张量

```
import torch

x = torch.tensor([[1., 2, 3], [4, 5, 6]])
z = torch.zeros(4, 8)
o = torch.ones(4, 8)
r = torch.randn(4, 8)
e = torch.empty(4, 8)   # 分配内存，但不初始化

print(x.shape)   # torch.Size([2, 3])
print(z.dtype)   # torch.float32
```

常见创建方式含义：

- `torch.tensor(...)`：从已有数据构造
- `torch.zeros(...)`：全 0
- `torch.ones(...)`：全 1
- `torch.randn(...)`：高斯随机初始化
- `torch.empty(...)`：只分配，不填值

3.2 `torch.empty` 为什么有用

初学者经常觉得 `empty` 很危险，确实如此，但它在工程上有明确用途：

- 你只想先拿到一块内存

- 稍后用自定义初始化逻辑填充
- 避免先写入无用的默认值

例如讲义里：

```
from torch import nn

x = torch.empty(4, 8)
nn.init.trunc_normal_(x, mean=0, std=1, a=-2, b=2)
```

这里的思路是：

1. 先拿到一块 4x8 的内存
2. 再用截断正态分布初始化

3.3 训练里最常关心的四个属性

```
x = torch.randn(2, 3, dtype=torch.float32, device="cpu")

print(x.shape)          # 形状
print(x.dtype)          # 数据类型
print(x.device)         # 存储设备
print(x.requires_grad) # 是否需要梯度
```

把这四个概念分清非常重要：

属性	含义	例子
`shape`	张量每一维大小	`(B, L, D)`
`dtype`	每个元素如何编码	`float32` 、 `bfloat16`
`device`	存在哪个设备上	`cpu` 、 `cuda:0`
`requires_grad`	是否参与梯度计算	参数通常是 `True`

4. 内存核算：dtype、参数、梯度、激活值到底占多少

4.1 一个 tensor 占多少内存

最基本公式：

$$\text{Memory} = \text{numel} \times \text{element_size}$$

对应到 PyTorch：

```
import torch

x = torch.zeros(4, 8)  # 默认 float32

print(x.numel())         # 32 个元素
print(x.element_size())  # 每个元素 4 bytes
print(x.numel() * x.element_size()) # 128 bytes
```

这就是原讲义中的辅助函数：

```
def get_memory_usage(x: torch.Tensor):
    return x.numel() * x.element_size()
```

4.2 常见 dtype 对比

dtype	每元素字节数	特点	训练中的地位
`float32`	4 bytes	精度高，动态范围大	稳定但贵
`float16`	2 bytes	省内存、快，但容易 underflow	需要小心使用
`bfloat16`	2 bytes	动态范围接近 fp32	现代训练非常常见
`fp8`	1 byte	更省更快，但风险更高	需要专门硬件和框架支持

4.3 为什么 `float16` 可能不稳定

讲义里用了一个很直接的例子：

```
x = torch.tensor([1e-8], dtype=torch.float16)
print(x)  # 可能变成 0，发生 underflow
```

原因是：

- `float16` 的可表示范围更窄
- 特别小的数可能直接下溢成 0
- 训练时这会让梯度、激活值或更新变得不稳定

4.4 为什么 `bfloat16` 这么重要

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
print(x)  # 不会像 float16 那样轻易变成 0
```

`bfloat16` 的关键优势不是“更精确”，而是：

- 占用和 `float16` 一样，都是 2 bytes
- 动态范围接近 `float32`

这意味着它在深度学习里通常是一个很好的折中：

- 比 fp32 省显存
- 比 fp16 更稳

4.5 一个大矩阵为什么会这么贵

讲义里举了 GPT-3 前馈层中一块矩阵的例子：

```
x = torch.empty(12288 * 4, 12288)
```

它的元素数是：

$$(12288 \times 4) \times 12288$$

如果是 `float32` ，总内存约 2.3 GB。

这说明一个关键事实：

> 大模型训练时，单个大矩阵本身就可能是 GB 级别的对象。

4.6 训练时真正占显存的是哪些东西

很多同学一开始只盯着“参数量”，但训练显存通常至少包含：

- 参数 `parameters`
- 梯度 `gradients`
- 优化器状态 `optimizer state`
- 激活值 `activations`

以 Adam 类优化器为例，经常需要为每个参数维护额外统计量，因此优化器状态可能和参数本体一样大，甚至更大。

一个非常常见的心智模型：

项目	是否与参数量同阶
参数	是
梯度	是
优化器状态	是，甚至可能是 2 倍参数量
激活值	与 batch size、sequence length、层数强相关

4.7 一个最小内存核算例子

下面把讲义中的思路写成一个更完整的例子：

```python
import torch
from torch import nn

B = 2           # batch size
D = 4           # hidden dim
num_layers = 2

num_parameters = D * D * num_layers + D
num_activations = B * D * num_layers
num_gradients = num_parameters
num_optimizer_states = num_parameters

total_memory_bytes = 4 * (
    num_parameters +
    num_activations +
    num_gradients +
    num_optimizer_states
)

print("parameters:", num_parameters)
print("activations:", num_activations)
print("gradients:", num_gradients)
print("optimizer states:", num_optimizer_states)
print("total bytes:", total_memory_bytes)
```

这个例子假设全部使用 `float32` ，所以乘以 4。

5. 张量存储与视图：stride、view、transpose、contiguous

这一部分是很多 PyTorch 初学者最容易“能写代码但没真正理解”的地方。

5.1 张量不只是一个数字盒子

PyTorch tensor 本质上可以理解为：

- 一段底层存储空间
- 外加一组元数据，告诉你如何解释这段存储

这些元数据包括：

- shape
- stride
- dtype
- storage offset

5.2 什么是 stride

看一个 4x4 张量：

```
import torch

x = torch.tensor([
    [0., 1, 2, 3],
    [4, 5, 6, 7],
    [8, 9, 10, 11],
    [12, 13, 14, 15],
])

print(x.stride())   # (4, 1)
```

含义是：

- 沿着第 0 维前进一步，要在底层存储中跳过 4 个元素
- 沿着第 1 维前进一步，只要跳过 1 个元素

手动计算 `x[1, 2]` 在存储中的索引：

```
r, c = 1, 2
index = r * x.stride(0) + c * x.stride(1)
print(index)  # 6
```

5.3 view：很多操作其实没有复制数据

讲义里反复强调了一个关键观点：

> 很多 tensor 操作只是创建一个新的“视图”，并没有复制底层存储。

例如：

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])

y1 = x[0]
y2 = x[:, 1]
y3 = x.view(3, 2)
y4 = x.transpose(1, 0)
```

这些操作通常都只是“换一种方式看同一块数据”。

5.4 如何验证两个 tensor 共用同一块存储

原讲义提供了这个辅助函数：

```
def same_storage(x: torch.Tensor, y: torch.Tensor):
    return x.untyped_storage().data_ptr() == y.untyped_storage().data_ptr()
```

使用示例：

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])
y = x[:, 1]

print(same_storage(x, y))  # True
```

5.5 为什么修改一个 view 会影响原张量

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])
y = x.transpose(1, 0)

x[0][0] = 100
print(y[0][0])  # 100
```

因为 `x` 和 `y` 指向的是同一块底层存储。

5.6 什么是 non-contiguous，为什么 `view` 有时会报错

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])
y = x.transpose(1, 0)

print(y.is_contiguous())  # False
y.view(2, 3)              # 可能报错
```

为什么？

- `transpose` 只是改了访问方式
- 改完之后，元素在逻辑上相邻，但在内存里不一定连续
- `view` 需要张量在内存上满足特定连续布局

所以原讲义给出的处理方式是：

```
y = x.transpose(1, 0).contiguous().view(2, 3)
```

这一步的含义是：

1. 先真的复制出一份连续内存
2. 再把它 reshape 成目标形状

5.7 工程直觉

这一节真正要记住的不是 API，而是代价差异：

- `view` 类操作通常很便宜
- 复制会带来额外内存和额外计算

这在大模型训练里非常重要，因为无意识的复制经常导致：

- 显存突然增加
- 吞吐下降
- 程序变慢但你还不容易发现

6. 计算核算：为什么矩阵乘法主导训练成本

6.1 张量操作的三种常见层次

原讲义把操作分成了几类：

1. 切片 / 视图类操作
2. 逐元素操作
3. 矩阵乘法

其中最贵的通常是矩阵乘法。

6.2 逐元素操作

```
x = torch.tensor([1, 4, 9])

print(x.pow(2))   # [1, 16, 81]
print(x.sqrt())   # [1, 2, 3]
print(x.rsqrt())  # [1, 1/2, 1/3]
print(x + x)      # [2, 8, 18]
print(x * 2)      # [2, 8, 18]
```

它们的成本一般与元素个数成正比：

$$O(mn)$$

6.3 一个有代表性的例子：上三角 mask

```
x = torch.ones(3, 3).triu()
print(x)
```

输出：

```
tensor([
    [1., 1., 1.],
    [0., 1., 1.],
    [0., 0., 1.]
])
```

这在因果注意力里非常常见，因为需要阻止位置 `i` 看到未来位置。

6.4 矩阵乘法为什么这么核心

```
x = torch.ones(16, 32)
w = torch.ones(32, 2)
y = x @ w

print(y.shape)  # [16, 2]
```

在深度学习里，大部分主要计算都会落到类似的模式：

- 线性层
- 注意力中的投影
- MLP
- 梯度计算中的反向 matmul

6.5 batched matmul

原讲义还演示了更高维的情形：

```
x = torch.ones(4, 8, 16, 32)
w = torch.ones(32, 2)
y = x @ w

print(y.shape)  # [4, 8, 16, 2]
```

这说明：

- 前面的维度通常可以看成 batch-like 维度
- 最后的两个维度参与矩阵乘法

6.6 矩阵乘法 FLOPs 公式

如果：

- `x` 形状是 `(B, D)`
- `w` 形状是 `(D, K)`
- 输出 `y` 形状是 `(B, K)`

那么每个输出元素都需要：

- `D` 次乘法

- `D-1` 次加法

在估算里通常记为：

$$2BDK$$

这是讲义中最重要的公式之一。

6.7 一个线性模型的 FLOPs 例子

```
B = 1024
D = 256
K = 64

num_flops = 2 * B * D * K
print(num_flops)
```

这里的含义可以理解成：

- `B`：数据点个数
- `D * K`：参数量级
- 前向 FLOPs 约为 `2 * 数据量 * 参数量`

6.8 FLOPs 和 FLOP/s 不是一回事

很多人第一次看这两个缩写会混。

写法	含义
`FLOPs`	完成一次计算总共做了多少浮点操作
`FLOP/s`	硬件每秒能做多少浮点操作

一个是“工作量”，一个是“速度”。

6.9 MFU：模型到底把硬件吃满了没有

原讲义定义：

$$\mathrm{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}}$$

其中：

- `actual FLOP/s` ：你实际测出来的吞吐
- `promised FLOP/s` ：硬件规格表给出的理论峰值

MFU 的意义：

- 接近 1：说明你非常接近硬件上限
- 低很多：说明还有通信、访存、kernel 设计、batch 太小等问题

讲义里给出的经验是：

> `MFU >= 0.5` 往往已经算不错。

6.10 为什么 dtype 会影响吞吐

讲义里专门比较了 `float32` 和 `bfloat16` 。

原因很直接：

- 低精度意味着更少内存带宽压力
- 硬件往往对低精度 Tensor Core 做了专门优化

因此在现代 GPU 上，经常会看到：

- `bfloat16` 实际吞吐明显高于 `float32`

7. `einops` 与 `jaxtyping` ：让高维张量代码更可读

这一节不是“必须会”，但在课程和工程里都很有价值，因为高维张量代码最怕维度记混。

7.1 `jaxtyping` ：把维度名字写进类型注解

讲义中的例子：

```
from jaxtyping import Float
import torch

x: Float[torch.Tensor, "batch seq heads hidden"] = torch.ones(2, 2, 1, 3)
```

它的作用主要是文档化：

- 让你知道每一维语义是什么
- 降低“这个 `-2` 、 `-1` 到底是哪两维”的混乱

注意：

> 在这份讲义语境里，它主要是帮助阅读，不是强运行时约束。

7.2 `einsum` ：更清楚地写广义矩阵乘法

传统写法：

```
x = torch.ones(2, 3, 4)  # batch, seq1, hidden
y = torch.ones(2, 3, 4)  # batch, seq2, hidden
z = x @ y.transpose(-2, -1)
```

`einops` 风格写法：

```
from einops import einsum

z = einsum(
    x, y,
    "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2"
)
```

这个版本的优点是：

- 每一维名字都写出来了
- 哪一维被求和一眼就能看懂
- 不用死记 `-2` 、 `-1`

7.3 `...` 的用法

```
z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
```

这里的 `...` 表示：

- 前面可以有任意多个公共维度
- 它们自动广播或保留

7.4 `reduce`：对高维张量做规约

传统写法：

```
y = x.mean(dim=-1)
```

`einops` 写法：

```
from einops import reduce

y = reduce(x, "... hidden -> ...", "sum")
```

优点仍然是语义清晰。

7.5 `rearrange`：拆维与并维

讲义里的例子非常典型，尤其是多头注意力里经常出现：

```
from einops import rearrange, einsum
import torch

x = torch.ones(2, 3, 8)  # batch, seq, total_hidden
w = torch.ones(4, 4)

x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
```

这个过程对应的思想是：

1. 把压平的 `total_hidden` 拆成 `heads * hidden1`
2. 在每个 head 内部做变换
3. 再把 head 维并回去

7.6 这一节真正要带走什么

如果你以后写 Transformer 代码，`einops` 的真正价值是：

- 降低维度 bug
- 让代码可审查

- 让你和队友能看懂张量在干什么

8. 自动求导与反向传播： `loss.backward()` 到底做了什么

8.1 从一个最简单的例子开始

讲义里的标量损失例子：

```
import torch

x = torch.tensor([1., 2, 3])
w = torch.tensor([1., 1, 1], requires_grad=True)

pred_y = x @ w
loss = 0.5 * (pred_y - 5).pow(2)
loss.backward()

print(w.grad)  # tensor([1., 2., 3.])
```

我们来解释一下：

- `pred_y = x @ w = 6`
- `loss = 0.5 * (6 - 5)^2 = 0.5`
- 对 `w` 的梯度是：

$$\frac{\partial \text{loss}}{\partial w} = (pred_y - 5) \cdot x = 1 \cdot x = [1, 2, 3]$$

这和代码输出一致。

8.2 为什么只有 `w.grad` 有值

在这个例子里：

- `w.requires_grad=True`
- `x` 没有要求梯度
- `loss` 和中间变量默认不会把 `.grad` 存起来

这也是原讲义里几行断言的含义：

```
assert loss.grad is None
assert pred_y.grad is None
assert x.grad is None
```

8.3 前向和反向谁更贵

对于线性层，讲义给出了非常重要的结论：

- 前向：大约 `2 * 数据量 * 参数量`
- 反向：大约 `4 * 数据量 * 参数量`
- 总计：大约 `6 * 数据量 * 参数量`

这就是为什么训练 FLOPs 常写成：

$$6 \times \text{tokens} \times \text{parameters}$$

8.4 两层线性模型的反向传播为什么更贵

讲义考虑了：

```
x --w1--> h1 --w2--> h2 -> loss
```

前向的两个矩阵乘法：

- `x @ w1`
- `h1 @ w2`

反向时你至少还要算：

- `w2.grad`
- `h1.grad`
- `w1.grad`

本质上又回到了矩阵乘法。

所以这节真正要记住的是：

> 反向传播不是“顺手一算”，它本身就是一大堆高成本线性代数运算。

9. 模型与参数： `nn.Parameter` 、初始化、自定义 `nn.Module`

9.1 参数在 PyTorch 里是什么

讲义直接用了：

```
from torch import nn
import torch

w = nn.Parameter(torch.randn(16384, 32))
print(isinstance(w, torch.Tensor))  # True
```

理解方式：

- `nn.Parameter` 本质上仍然是 tensor
- 但它会被 `nn.Module` 自动识别为“这是需要训练的参数”

9.2 为什么初始化不能乱来

如果你直接写：

```
w = nn.Parameter(torch.randn(input_dim, output_dim))
x = nn.Parameter(torch.randn(input_dim))
output = x @ w
```

当 `input_dim` 很大时，输出幅值往往也会跟着变大。

这会带来两个问题：

- 前向数值容易爆
- 反向梯度也容易不稳定

9.3 Xavier 风格初始化的核心思想

讲义里的处理方式是：

```python
import numpy as np
from torch import nn
import torch

input_dim = 16384
output_dim = 32

w = nn.Parameter(torch.randn(input_dim, output_dim) / np.sqrt(input_dim))
```

核心思想：

- 按输入维度缩放参数标准差
- 让输出量级不要随着输入维度无限增大

进一步，讲义还用了截断正态分布：

```python
w = nn.Parameter(
    nn.init.trunc_normal_(
        torch.empty(input_dim, output_dim),
        std=1 / np.sqrt(input_dim),
        a=-3, b=3
    )
)
```

目的很明确：

- 保留合理的尺度
- 尽量避免极端离群值

9.4 自定义一个简单线性层

讲义自己写了一个最小版本的 `Linear`：

```python
import numpy as np
import torch
from torch import nn

class Linear(nn.Module):
    def __init__(self, input_dim: int, output_dim: int):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(input_dim, output_dim) / np.sqrt(input_dim)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x @ self.weight
```

这个例子非常值得看，因为它说明：

- `nn.Linear` 不是魔法
- 本质上就是参数 + 前向函数

9.5 自定义多层模型 `Cruncher`

讲义进一步搭了一个简单深层线性模型：

```python
class Cruncher(nn.Module):
    def __init__(self, dim: int, num_layers: int):
        super().__init__()
        self.layers = nn.ModuleList([
            Linear(dim, dim)
            for _ in range(num_layers)
        ])
        self.final = Linear(dim, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, D = x.size()
        for layer in self.layers:
            x = layer(x)
        x = self.final(x)
        x = x.squeeze(-1)
        assert x.size() == torch.Size([B])
        return x
```

这段代码有三个学习点：

1. `nn.ModuleList` 用来注册一组子模块
2. `forward` 里按层顺序执行
3. 最后 `squeeze(-1)` 把 `[B, 1]` 变成 `[B]`

9.6 如何统计参数量

讲义里的函数：

```
def get_num_parameters(model: nn.Module) -> int:
    return sum(param.numel() for param in model.parameters())
```

这是训练前几乎必做的一件事，因为很多资源估算都依赖参数量。

9.7 为什么模型也要 `.to(device)`

很多同学一开始只把输入移到 GPU，却忘了模型参数也在 CPU。

正确做法：

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = model.to(device)
x = torch.randn(8, 64, device=device)
y = model(x)
```

如果参数和输入不在同一设备上，程序会直接报错。

10. 训练工程基础：随机性、数据加载、优化器、训练循环

这一部分是把前面所有底层概念真正串起来的地方。

10.1 随机性控制为什么重要

讲义提醒了一个很好的工程习惯：

> 调试时，最好让每次运行尽量可复现。

最基本的做法：

```python
import torch
import numpy as np
import random

seed = 0
torch.manual_seed(seed)
np.random.seed(seed)
random.seed(seed)
```

随机性会出现在：

- 参数初始化
- dropout
- 数据打乱
- batch 采样

如果 seed 不固定，debug 会困难很多。

10.2 语言模型数据为什么常用 `memmap`

讲义里的动机很直接：

- 数据本质上是一长串 token id
- 数据集可能大到不能整体加载进内存
- 于是可以使用 `numpy.memmap` 按需访问

示例：

```python
import numpy as np

orig_data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=np.int32)
orig_data.tofile("data.npy")

data = np.memmap("data.npy", dtype=np.int32)
print(data[:5])
```

这里要理解的是思想，而不是这个玩具数据本身：

- 数据文件在磁盘
- 你只访问当前需要的一段
- 不必一次性把所有 token 全读进 RAM

10.3 `get_batch` 做了什么

讲义的 `get_batch` 很值得逐行读：

```
def get_batch(data, batch_size, sequence_length, device):
    start_indices = torch.randint(len(data) - sequence_length, (batch_size,))
    x = torch.tensor([
        data[start:start + sequence_length]
        for start in start_indices
    ])
    if torch.cuda.is_available():
        x = x.pin_memory()
    x = x.to(device, non_blocking=True)
    return x
```

它完成了四件事：

1. 随机抽取多个起点
2. 从长序列中切出多个长度为 `sequence_length` 的片段
3. 可选地把 CPU 内存 pin 住
4. 把 batch 异步搬到 GPU

10.4 什么是 pinned memory

原讲义强调：

- 普通 CPU tensor 默认在 paged memory 中
- `pin_memory()` 会把它放到 pinned memory
- 这样拷到 GPU 时可以更高效，也更容易与计算并行

```
if torch.cuda.is_available():
    x = x.pin_memory()

x = x.to(device, non_blocking=True)
```

这里的工程意图是：

- 当前 batch 正在 GPU 上跑
- 下一批数据可以同时在 CPU 侧准备和传输

10.5 自己写一个最小 SGD

讲义没有直接用现成 `torch.optim.SGD`，而是自己实现了一版：

```
class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=0.01):
        super().__init__(params, dict(lr=lr))

    def step(self):
        for group in self.param_groups:
            lr = group["lr"]
            for p in group["params"]:
                grad = p.grad.data
                p.data -= lr * grad
```

这个版本很朴素，但非常有教学价值，因为它把更新规则完全暴露出来了：

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

10.6 AdaGrad 的核心思想

讲义接着自己实现了 AdaGrad：

```
class AdaGrad(torch.optim.Optimizer):
    def __init__(self, params, lr=0.01):
        super().__init__(params, dict(lr=lr))

    def step(self):
        for group in self.param_groups:
            lr = group["lr"]
            for p in group["params"]:
                state = self.state[p]
                grad = p.grad.data
                g2 = state.get("g2", torch.zeros_like(grad))
                g2 += torch.square(grad)
                state["g2"] = g2
                p.data -= lr * grad / torch.sqrt(g2 + 1e-5)
```

你可以把它理解为：

- 不只是看当前梯度
- 还看历史梯度平方和

- 对不同参数使用不同尺度的更新

讲义还顺带串起了优化器家族关系：

- momentum = SGD + 梯度指数滑动平均
- AdaGrad = SGD + 梯度平方累计
- RMSProp = AdaGrad + 指数衰减
- Adam = RMSProp + momentum

10.7 一个完整训练 step 到底做了什么

原讲义的训练闭环可以概括成四步：

```
x, y = get_batch(...)
pred_y = model(x)
loss = F.mse_loss(pred_y, y)
loss.backward()
optimizer.step()
optimizer.zero_grad(set_to_none=True)
```

更完整地说：

1. 取数据
2. 前向计算预测值
3. 计算 loss
4. 反向传播得到梯度
5. 优化器根据梯度更新参数
6. 清空梯度，准备下一步

10.8 为什么要 `zero_grad`

PyTorch 默认会累加梯度，而不是自动覆盖。

所以如果你不清：

- 第 2 个 step 的梯度会加到第 1 个 step 上
- 训练行为就变了

讲义中用了：

```
optimizer.zero_grad(set_to_none=True)
```

`set_to_none=True` 的好处通常是：

- 更省一点内存
- 让后续框架逻辑更清楚地区分“还没算过梯度”和“梯度是 0”

10.9 讲义中的最小训练实验

原讲义构造了一个线性真值：

```
D = 16
true_w = torch.arange(D, dtype=torch.float32, device=get_device())
```

然后随机生成输入 `x`，令目标为：

```
true_y = x @ true_w
```

这等于人为制造了一个“真实线性规律”，再让模型去学它。

这是非常好的教学套路，因为：

- 数据分布清楚
- 目标函数清楚
- 很容易观察训练有没有收敛

11. checkpoint 与混合精度训练

11.1 为什么训练一定要做 checkpoint

讲义里一句话说得很实在：

> 训练语言模型要跑很久，而且几乎肯定会崩一次。

所以 checkpoint 不是锦上添花，而是必需品。

最基本的保存方式：

```
checkpoint = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
}
torch.save(checkpoint, "model_checkpoint.pt")
```

恢复方式：

```
loaded_checkpoint = torch.load("model_checkpoint.pt")
```

真实训练中通常还会额外保存：

- 当前 step
- 学习率调度器状态
- 随机种子状态
- scaler 状态（如果用了 AMP）

11.2 `state_dict` 到底是什么

可以把它理解成：

- 模型参数和 buffer 的一个字典
- 优化器内部状态的一个字典

它是 PyTorch 里最标准的序列化接口。

11.3 混合精度训练为什么是现代训练的默认选项

讲义对不同精度的 trade-off 总结得很明确：

- 高精度：更稳，但更慢、更占内存
- 低精度：更省、更快，但更容易数值不稳定

所以混合精度训练的目标就是：

> 在真正需要稳定性的地方保留高精度，在可以接受的地方使用低精度。

11.4 讲义里的混合精度思路

原讲义把核心想法概括成：

- 前向尽量使用 `bfloat16` 或 `fp8`
- 参数、梯度等关键部分保留 `float32`

这是“同时拿到速度和稳定性”的经典工程折中。

11.5 为什么现在很多训练偏爱 `bfloat16`

结合前面 dtype 一节，可以总结为：

- 显存更省
- Tensor Core 吞吐更高
- 动态范围更大，不像 `float16` 那么容易 underflow

这也是为什么现代大模型训练里经常看到 `bf16` 成为默认选择。

11.6 AMP 的角色

讲义里提到 PyTorch 提供了 AMP：

- 自动混合精度
- 自动为适合的算子选择较低精度执行

虽然本讲没有展开代码细节，但你应该记住它在训练栈中的定位：

- 它不是新的模型结构
- 它是一套数值与硬件协同优化机制

12. 把整节课串起来：一个最小训练系统是如何工作的

如果把这节课所有内容串成一条线，它其实描述了这样一个系统：

1. 用 tensor 表示数据和参数
2. 用合适的 dtype 和 device 放置它们
3. 通过矩阵乘法和逐元素操作完成前向计算
4. 用 autograd 自动计算梯度
5. 用优化器根据梯度更新参数
6. 用数据加载逻辑持续提供 batch
7. 用 checkpoint 保住长时训练结果
8. 用 mixed precision 在速度、显存、稳定性之间做平衡

这就是一个深度学习训练系统最小但完整的骨架。

13. 高频考点与易错点

13.1 高频结论

- tensor 内存 = `numel * element_size`
- `float32` 是 4 bytes， `float16` 和 `bfloat16` 是 2 bytes
- 大矩阵乘法 FLOPs 约为 `2mnp`
- 一步训练常用估算： `forward + backward ≈ 6 * 数据量 * 参数量`
- `view` 通常不复制数据， `contiguous()` 通常会复制
- `bfloat16` 常比 `float16` 更适合训练
- 优化器状态会明显增加总显存

13.2 高频误区

误区 1：参数量就是显存占用

错。训练显存至少还包括梯度、优化器状态、激活值。

误区 2： `transpose` 之后想怎么 `view` 都可以

错。 `transpose` 后张量常常变成 non-contiguous，需要先 `contiguous()` 。

误区 3： `loss.backward()` 很便宜

错。反向传播本身也是高成本矩阵计算。

误区 4：低精度一定更好

错。低精度是拿数值稳定性去换速度和显存，必须配合具体硬件和训练策略使用。

误区 5：不写 `zero_grad()` 也没事

错。PyTorch 默认累积梯度，不清空会改变训练行为。

14. 复习清单

如果你复习完这份讲义，至少应该能独立回答下面这些问题：

- 为什么训练系统里最重要的两个资源是内存和计算
- 如何从 `shape` 和 `dtype` 估算一个 tensor 的内存占用
- `float16` 和 `bfloat16` 的关键差别是什么
- 为什么 view 操作便宜，而 copy 操作昂贵
- 为什么矩阵乘法在训练中通常是主导成本
- `2BDK` 这个 FLOPs 公式是怎么来的
- 为什么训练一步常常估算成 `6 * 数据量 * 参数量`
- `nn.Parameter` 和普通 tensor 的区别是什么
- 为什么初始化要按 `1 / sqrt(input_dim)` 缩放
- 一个训练 step 的标准顺序是什么
- 为什么要做 checkpoint
- 为什么现代训练大量使用 mixed precision

15. 附：可运行的最小整合示例

下面给出一个把本讲多个核心点串在一起的最小示例。

```python
import torch
import torch.nn.functional as F
import numpy as np
from torch import nn


def get_device():
    return torch.device("cuda:0" if torch.cuda.is_available() else "cpu")


class Linear(nn.Module):
    def __init__(self, input_dim: int, output_dim: int):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(input_dim, output_dim) / np.sqrt(input_dim)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x @ self.weight


class Cruncher(nn.Module):
    def __init__(self, dim: int, num_layers: int):
        super().__init__()
        self.layers = nn.ModuleList([Linear(dim, dim) for _ in range(num_layers)])
        self.final = Linear(dim, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        for layer in self.layers:
            x = layer(x)
        x = self.final(x).squeeze(-1)
        return x


class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=0.01):
        super().__init__(params, dict(lr=lr))

    def step(self):
        for group in self.param_groups:
            lr = group["lr"]
            for p in group["params"]:
                if p.grad is not None:
```

```
                    p.data -= lr * p.grad.data


def get_batch(B: int, D: int, device: torch.device):
    true_w = torch.arange(D, dtype=torch.float32, device=device)
    x = torch.randn(B, D, device=device)
    y = x @ true_w
    return x, y


device = get_device()
model = Cruncher(dim=16, num_layers=2).to(device)
optimizer = SGD(model.parameters(), lr=0.01)

for step in range(10):
    x, y = get_batch(B=8, D=16, device=device)
    pred_y = model(x)
    loss = F.mse_loss(pred_y, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
    print(f"step={step:02d}, loss={loss.item():.4f}")
```

你可以用这个例子检查自己是否真正理解了：

- 参数如何定义
- 前向如何发生
- 梯度如何产生
- 参数如何更新
- 一个训练循环如何闭环

16. 一句话总结构图

如果只用一句话概括这节课：

> 深度学习训练的本质，就是在有限的内存和算力预算下，用张量和矩阵运算组织前向、反向、更新，并尽可能高效地把整个循环持续跑起来。